

Relatório de Integração — Sprint 2 (MOM)

Projeto: FieldFlow — Marketplace de Serviços Agrícolas **Aluno:** Rafael Moura Ganascini · **Disciplina:** LDAMD — PUC Minas, 2026/1

Escolha da ferramenta

Adotei o **RabbitMQ 4.3.0** como MOM. Kafka foi descartado por complexidade operacional desnecessária para um projeto acadêmico. Redis Pub/Sub é *fire-and-forget* sem persistência — inaceitável para eventos como "demanda criada" ou "perfil enviado para análise". RabbitMQ entrega broker AMQP maduro, filas duráveis, mensagens persistentes, *routing* declarativo por padrão e UI de gerenciamento que facilita evidenciar o funcionamento.

Padrão arquitetural

Publish/Subscribe com topic exchange. A API não sabe quem processa os eventos — ela só publica no exchange `fieldflow.events` usando uma chave de roteamento no formato `<recurso>.<acao>[.<detalhe>]` (ex.: `demanda.status.aceito`). O worker é um container separado que se inscreve nos padrões que interessam (`demanda.#`, `usuario.#`, `prestador.#`, `candidatura.#`) e cuida do processamento. Para adicionar um novo consumidor — envio de e-mail, indexador de busca, etc. — basta subir outro processo inscrito; produtor e consumidores ficam desacoplados.

Casos de negócio assíncronos

Dois fluxos exemplificam a vantagem de EDA sobre orquestração síncrona:

- Validação de perfil do prestador** — `POST /prestadores/me/perfil` persiste com status `EM_ANALISE`, publica `prestador.perfil.enviado` e retorna 200 imediatamente. O worker valida (≥ 1 ano de experiência e ≥ 1 certificação), atualiza para `APROVADO/REPROVADO` e publica novo evento. Só prestadores `APROVADO` podem se candidatar.
- Rejeição em cascata** — quando o cliente aceita uma candidatura (`POST /candidaturas/{id}/aceitar`), a API publica `candidatura.aceita` + `demanda.status.aceito` e devolve 200. O worker recebe `candidatura.aceita` e marca todas as outras candidaturas `PENDENTES` da mesma demanda como `REJEITADA`, publicando um `candidatura.rejeitada` por concorrente. Caso textbook de "um evento dispara N reações" — o cliente não precisa esperar a API processar cada rejeição.

Comunicação 100% assíncrona



Produtor e consumidor **não trocam HTTP entre si**. `GET /notificacoes` é só uma janela de leitura sobre o que o worker gravou.

Desafios e mitigações

- Ordem de inicialização:** `depends_on: condition: service_healthy` + `retry loop` no worker (`_connect_with_retry, 30x2s`).
- Acoplamento em testes:** `NoopEventPublisher` selecionado em runtime conforme presença de `RABBITMQ_URL`.
- Worker como produtor:** validação publica `prestador.aprovado/reprovado` reusando o mesmo `EventPublisher singleton`.
- Idempotência:** cada `publish()` gera `event_id` (UUID) no `message_id` AMQP + payload; `notificacoes` tem `UNIQUE(event_id)` e o consumer pula duplicados — resolve `redelivery`.
- Dead-letter queue:** exchange `fieldflow.events.dlx` (fanout) + fila `fieldflow.notificacoes.dlq`; falhas vão pra DLQ via `nack(requeue=False)`. **Reprocessamento manual** via `python -m worker reprocess-dlq`: drena a DLQ, lê a routing key original do header `x-death` e republica no exchange principal (dedup garantida pelo `UNIQUE(event_id)` em `notificacoes`).
- Migrações versionadas:** schema gerenciado por **Alembic** (baseline em `0001_baseline`). O startup da API e do worker chama `alembic upgrade head` programaticamente — mudanças de schema viram novas revisões, sem `docker compose down -v`.
- Dados sensíveis:** senha (texto ou hash) **nunca** integra payload de evento.

Limites e próximos passos

Permanece como dívida assumida o **outbox pattern**: uma falha entre o commit no banco e o `basic_publish` ainda pode gerar inconsistência (a API responde sucesso, mas o evento nunca chega no broker). Solução planejada: tabela `outbox` escrita na mesma transação do agregado + relay separado publicando dela. Candidato a Sprint 3.